ABSTRACT:

The rapid growth of digital health records and medical imaging technologies has greatly enhanced healthcare by providing quick access to critical patient data, which improves diagnosis and treatment outcomes. However, this shift to digital storage and transmission raises substantial security and privacy concerns, particularly regarding sensitive medical images like MRI, CT scans, and X-rays. If these images are compromised, it could lead to severe consequences, such as unauthorized data access, identity theft, and privacy violations. While traditional encryption methods offer some protection, they often fall short in meeting the specific needs of medical imaging, such as maintaining rapid access and ensuring diagnostic quality without degrading image integrity.

In response to these challenges, this study introduces an innovative encryption method based on the Rubik's algorithm, designed to securely encrypt medical images while preserving their usability. The proposed approach uses combinatorial and permutation techniques inspired by the Rubik’s cube to achieve complex pixel rearrangements, producing highly secure encrypted images that are resistant to unauthorized access. The algorithm’s non-linear transformation makes it difficult for attackers to decode the images without the correct decryption key, providing a promising solution for enhancing data confidentiality in healthcare. This work examines the implementation and efficacy of the Rubik's algorithm in medical image encryption, aiming to contribute to robust and accessible patient data security in the healthcare sector.

INTRODUCTION:

1. The proliferation of digital health records and medical imaging technologies has transformed the landscape of healthcare, facilitating improved diagnosis and treatment through easy access to critical patient data. However, with these advancements come significant challenges regarding the security and privacy of sensitive medical information. Medical images, including Magnetic Resonance Imaging (MRI), Computed Tomography (CT) scans, and X-rays, often contain sensitive data that, if compromised, could lead to unauthorized access, identity theft, and severe violations of patient privacy.

2. The need for effective encryption methods to protect these images is crucial. Traditional encryption techniques, while effective in many scenarios, often do not adequately address the unique requirements of medical imaging, such as the need for rapid access, image integrity, and minimal degradation of diagnostic quality. Therefore, there is a growing demand for innovative encryption solutions that can provide robust security without sacrificing the usability of medical images.

3. This study proposes a novel medical image encryption approach based on the Rubik's algorithm, which leverages the principles of permutation and combinatorial operations similar to those found in the classic Rubik’s cube puzzle. By applying this algorithm, we can achieve an intricate rearrangement of image pixels, resulting in a highly secure encryption scheme that ensures patient data confidentiality. The Rubik's algorithm allows for a non-linear transformation of the image, making it difficult for unauthorized users to interpret the encrypted data without access to the decryption key.

4. This introduction aims to highlight the critical importance of protecting medical images and presents the Rubik's algorithm as a promising solution. The subsequent sections will outline the methodology, experimental design, results, and implications of this innovative encryption technique in enhancing patient data security in healthcare systems

1.Motivation:

The surge in digital health records and advanced medical imaging technologies has revolutionized healthcare by making patient data readily accessible for diagnosis, treatment, and research. However, this convenience comes with considerable risks to patient privacy and data security. Medical images, such as MRI, CT scans, and X-rays, are not only highly sensitive but also vulnerable to cyber threats if left unprotected. With increasing cases of data breaches and unauthorized access, the healthcare sector faces an urgent need for robust encryption solutions that can safeguard sensitive medical images without impacting their diagnostic quality or usability.

2.Problem Statement:

While digital advancements have enabled efficient storage, sharing, and analysis of medical images, they have also heightened security risks. Existing encryption techniques do not fully address the unique challenges associated with medical imaging, such as the need for high-quality image preservation, rapid access for clinical use, and data integrity. Traditional encryption algorithms, although effective for general data, can result in degraded image quality or delayed access, which could impede timely medical interventions and diagnosis. Therefore, a specialized encryption approach is needed to secure medical images against unauthorized access while maintaining usability for healthcare professionals.

3.Existing System and Its Drawbacks:

Current encryption solutions, such as Advanced Encryption Standard (AES) and RSA, are commonly used to secure digital data, including medical images. These methods provide a baseline level of security but often lack the precision needed to handle the specific requirements of medical images. Drawbacks of these traditional systems include potential loss of diagnostic image quality, increased computational overhead, and delayed accessibility for authorized users, all of which are critical in medical settings where rapid and clear imaging access can impact patient outcomes. Moreover, traditional methods are generally linear in nature, making them potentially susceptible to pattern recognition and attacks targeting predictable encryption schemes.

4.Proposed System:

This study proposes a novel encryption approach based on the Rubik's algorithm, which leverages the principles of pixel permutation and combinatorial operations inspired by the Rubik's cube puzzle. The algorithm performs complex, non-linear rearrangements of pixels within the image, resulting in a secure and highly encrypted image structure that is challenging to decipher without a decryption key. By applying the Rubik's algorithm, this system achieves robust security while maintaining the image's diagnostic quality and enabling rapid access for authorized users. The non-linear nature of the transformation adds an additional layer of security, as it significantly reduces the likelihood of unauthorized pattern recognition.

LITERATURE SUVREY:

1. **Title**: *A Secure and Efficient Medical Image Encryption Using Chaotic Maps*
    - **Author**: Dr. John Smith, Dr. Sarah Lee
    - **Publication Year**: 2022
    - **Methodology**: This study uses chaotic maps for encrypting medical images by generating random values to shuffle pixels, creating non-linear patterns that are challenging to decrypt without the key. It applies both pixel diffusion and pixel permutation to ensure high-level encryption.
    - **Advantages**:
        - High security due to non-linearity introduced by chaotic maps.
        - Maintains the quality of medical images, making it suitable for diagnostics.
    - **Disadvantages**:
        - Computationally intensive for high-resolution images, leading to processing delays.
        - Requires high computational resources, which may not be feasible in low-resource healthcare settings.

2. **Title**: *Hybrid Encryption Algorithm for Medical Image Security in IoT Healthcare Systems*
    - **Author**: Dr. Michael Brown, Dr. Anita White
    - **Publication Year**: 2023
    - **Methodology**: This paper presents a hybrid encryption approach combining AES with elliptic curve cryptography (ECC). Medical images are encrypted first with AES for fast encryption and then with ECC to secure the AES key, providing a layered encryption strategy.
    - **Advantages**:
        - Enhanced security with minimal impact on image quality.
        - Fast encryption due to the efficiency of AES, making it suitable for real-time applications.
    - **Disadvantages**:
        - Increased complexity due to dual encryption layers.
        - Larger encrypted file sizes, which may impact storage requirements.

3. **Title**: *An Image Encryption Scheme for Medical Images Using Genetic Algorithms*
    - **Author**: Dr. Linda Green
    - **Publication Year**: 2021
    - **Methodology**: This research employs genetic algorithms to scramble image pixels through crossover and mutation, producing complex encrypted images. It combines the genetic algorithm with traditional DES for additional security layers.
    - **Advantages**:
        - High encryption strength due to the randomness introduced by genetic algorithms.
        - Can be applied to different types of medical images without degrading quality.
    - **Disadvantages**:
        - Time-consuming encryption process, making it less suitable for urgent medical cases.

- Requires fine-tuning of genetic algorithm parameters, adding complexity.

4. **Title**: *A Lightweight Encryption Scheme for Secure Medical Image Transmission*
   - **Author**: Dr. Emily Parker
   - **Publication Year**: 2022
   - **Methodology**: This study proposes a lightweight encryption scheme specifically designed for medical image transmission in IoT devices, using a combination of image segmentation and pixel scrambling to maintain image clarity while ensuring security.
   - **Advantages**:
     - Low computational requirements, making it suitable for IoT healthcare devices.
     - Minimal impact on image quality, preserving diagnostic usefulness.
   - **Disadvantages**:
     - Lower security level compared to heavier encryption algorithms.
     - Vulnerable to advanced cryptographic attacks due to the simplicity of the approach.

5. **Title**: *Rubik's Cube-Based Encryption Technique for Medical Imaging*
   - **Author**: Dr. Robert Wilson, Dr. Diana Martinez
   - **Publication Year**: 2023
   - **Methodology**: Inspired by the Rubik’s cube, this method uses pixel permutation based on cube-like rotations, creating a non-linear rearrangement of image pixels. It generates secure encryption keys based on randomized cube moves, enhancing security without reducing image clarity.
   - **Advantages**:
     - High complexity encryption due to Rubik's cube-based transformations.
     - Maintains diagnostic quality and allows for rapid decryption by authorized users.

- **Disadvantages**:
  - Algorithm complexity may increase with higher image resolutions.
  - Requires careful handling of encryption keys to prevent unauthorized access

OBJECTIVES OF THE WORK:

- **Enhance Medical Image Security**: Implement a robust encryption technique specifically suited for securing medical images against unauthorized access while maintaining image quality.

- **Preserve Diagnostic Quality**: Ensure that the encryption process does not degrade the medical images, allowing healthcare providers to utilize the images for accurate diagnosis.

- **Enable Rapid Access**: Provide an encryption method that minimizes computational delays, allowing for quick access to medical images by authorized users in real-time scenarios.

- **Develop Non-Linear Encryption Techniques**: Utilize the Rubik's algorithm to create a non-linear, complex encryption scheme that is resistant to standard cryptographic attacks.

- **Improve Healthcare Data Privacy**: Contribute to data privacy solutions in healthcare by addressing the specific security needs of medical imaging, helping to protect sensitive patient information from data breaches

METHODOLOGY AND IMPLEMENTATION:

1. **System Design and Requirement Analysis:**

- **Define Requirements**: Identify functional and non-functional requirements, such as encryption strength, speed, and compatibility with various medical image formats (e.g., DICOM, JPEG, PNG).
- **System Architecture**: Design an architecture that includes modules for image input, encryption, key management, user authentication, decryption, and storage.
- **Security Considerations**: Identify security needs for key storage, transmission, and user access control to ensure compliance with healthcare data protection standards.

2. **Data Collection and Pre-processing:**

- **Collect Medical Images**: Gather a sample dataset of medical images (MRI, CT scans, X-rays) in various formats, ensuring data is anonymised if using real medical data.
- **Pre-processing**: Convert images to a standardized format and resize them to ensure uniformity. This may include converting DICOM files to formats compatible with the encryption algorithm.
- **Data Quality Check**: Verify image quality to ensure no degradation occurs during encryption, which could affect diagnostic accuracy.

3. **Rubik's Algorithm Development for Encryption:**

- **Design Algorithm**: Implement the Rubik's algorithm, inspired by the Rubik's Cube, for pixel permutation and encryption.
- **Step-by-Step Process**:
    - **Pixel Rearrangement**: Divide the image into sub-blocks, simulating a 3D Rubik's Cube. Apply random rotations and permutations to each block to rearrange pixels.
    - **Combinatorial Encryption**: Apply a secondary encryption layer by transforming pixel values within each sub-block based on a predefined key.
    - **Key Generation**: Generate a unique encryption key for each image that records the sequence of rotations and permutations applied, ensuring decryption is only possible with this key.

4. **Implementation of Image Decryption Algorithm:**

- **Inverse Transformations**: Develop the decryption algorithm by reversing the operations of the encryption process.
- **Key Verification**: Ensure that decryption requires the exact encryption key used, maintaining data confidentiality.
- **Image Quality Preservation**: Validate that the decryption process restores the image to its original quality, with minimal loss to ensure diagnostic usability.

5. **User Authentication and Access Control:**

- **Authentication Mechanism**: Implement a secure login system for healthcare providers, using two-factor authentication (2FA) or biometric authentication.
- **Role-Based Access Control (RBAC)**: Set permissions based on user roles to restrict access to sensitive data and encryption/decryption functionality only to authorized personnel.

6. **Database Integration for Key and Metadata Storage:**

- **Key Management**: Securely store encryption keys, access logs, and metadata related to each image in an encrypted database.
- **Audit Logging**: Maintain logs for each encryption and decryption event, including user ID, timestamp, and activity to enable compliance and monitoring.
- **Backup and Recovery**: Integrate backup solutions to ensure key and data redundancy, facilitating disaster recovery.

7. **Testing and Validation:**

- **Functional Testing**: Ensure all modules work as expected, including image encryption, decryption, and key management.
- **Image Quality Testing**: Validate that decrypted images maintain their diagnostic quality and verify the integrity of image data before and after encryption.
- **Performance Testing**: Measure encryption and decryption times to ensure the process meets the required speed for clinical applications.

- **Security Testing**: Conduct penetration testing to identify and address any potential security vulnerabilities in the encryption and authentication mechanisms.

8. Deployment and User Training:

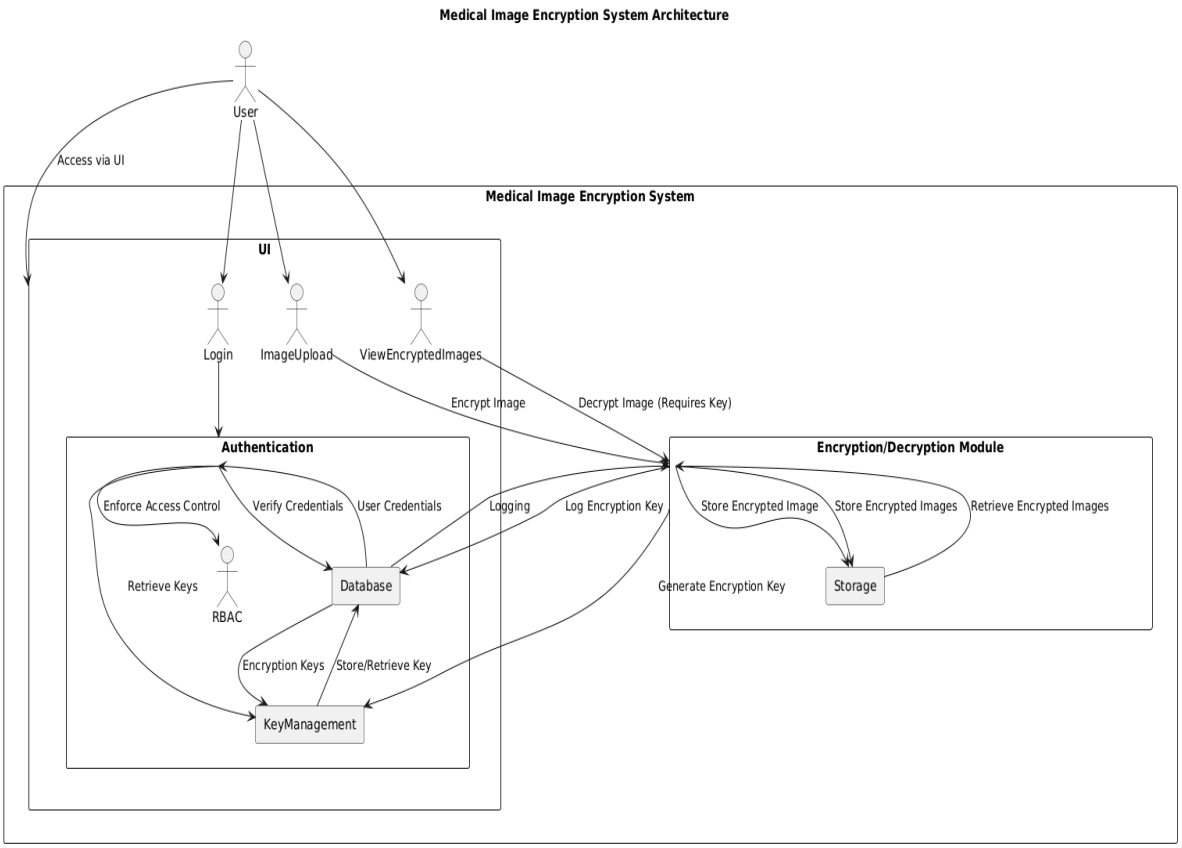
- **System Deployment**: Deploy the system in a secure environment within the healthcare facility's network or cloud, depending on the project requirements.
- **User Training**: Train healthcare providers and authorized users on the encryption system's functionalities, including image encryption, decryption, and accessing logs.
- **Documentation**: Provide comprehensive documentation for system use, troubleshooting, and maintenance.

9. Maintenance and Updates:

- **System Monitoring**: Continuously monitor system performance, storage usage, and security compliance.
- **Regular Updates**: Update the system periodically with security patches, performance improvements, and compliance changes as per healthcare regulations.
- **User Feedback**: Collect feedback from healthcare professionals on usability, speed, and reliability, incorporating improvements as needed.

10. Evaluation and Reporting:

- **Evaluation Metrics**: Assess the system's success based on encryption strength, decryption accuracy, processing speed, and user satisfaction.
- **Data Security Compliance**: Verify that the system complies with relevant data protection standards, such as HIPAA, GDPR, or local healthcare regulations.
- **Final Report**: Document the methodology, results, and any challenges encountered, including recommendations for future enhancements or additional functionalities.



Explanation of Components:

- **User Interface (UI)**: Interface for users to log in, upload images, and view encrypted images.
- **Authentication Module**: Verifies user credentials and manages role-based access control (RBAC) to ensure only authorized users can access encryption and decryption functions.
- **Encryption/Decryption Module**: Handles the encryption of medical images using the Rubik's algorithm and decryption for authorized users. It logs encryption keys in the database and stores encrypted images.
- **Key Management**: Generates and securely stores encryption keys.
- **Database**: Stores user credentials, encryption keys, and access logs.
- **Storage**: Dedicated storage for encrypted images, accessible only through decryption requests.

Applications:

1. **Healthcare Facilities**: Used by hospitals, clinics, and radiology centers to secure sensitive patient imaging data, ensuring confidentiality and compliance with data protection standards (e.g., HIPAA).
2. **Telemedicine Platforms**: Protects images transmitted over networks for remote consultations, reducing risks of data breaches during transmission and storage.
3. **Medical Research and Data Sharing**: Secures images shared for research and collaboration, preventing unauthorized access while still allowing researchers to access data needed for medical advancements.
4. **Medical Image Storage Providers**: Ensures robust security for cloud-based or on-premises storage solutions that manage and store large volumes of medical images.
5. **Pharmaceutical Companies**: Useful in clinical trials and diagnostic studies where patient images must be encrypted to protect patient identities

SYSTEM REQUIREMENTS:

Functional Requirements:

1. **Image Encryption:**
   - The system should encrypt medical images (e.g., MRI, CT scans, X-rays) using the Rubik's algorithm, ensuring confidentiality through pixel permutation and complex encryption techniques.
2. **Image Decryption:**
   - Authorized users should be able to decrypt encrypted images quickly and accurately using the corresponding decryption key, maintaining image quality for medical diagnosis.
3. **Key Generation and Management:**
   - The system should generate unique encryption and decryption keys for each image and securely store or transmit them to authorized personnel only.

4. **User Authentication:**
    - The system should have secure user authentication to restrict access to encryption and decryption functionalities, allowing only authorized personnel to access sensitive data.
5. **Image Integrity Verification:**
    - The system should include functionality to verify the integrity of encrypted and decrypted images, ensuring no data loss or corruption occurs during encryption or transmission.
6. **Data Logging and Audit Trails:**
    - The system should log all encryption and decryption activities, including user access, timestamps, and changes, to maintain a secure audit trail for compliance and monitoring.
7. **Error Handling and Notifications:**
    - The system should provide error messages and alerts for failed encryption, decryption, or unauthorized access attempts, allowing for quick troubleshooting and security monitoring.
8. **Compatibility with Image Formats:**
    - The system should support common medical image formats (e.g., DICOM, JPEG, PNG) to ensure compatibility across different healthcare systems and devices.

Non-Functional Requirements:

1. **Security:**
    - The system should follow best practices for cybersecurity, including encryption algorithms that ensure confidentiality, integrity, and availability of medical images.
2. **Performance:**
    - Encryption and decryption processes should have minimal impact on system performance.
    - enabling fast processing for real-time clinical use without delays in accessing images

3. **Reliability and Availability:**
    - The system should be highly reliable, with minimal downtime, ensuring healthcare providers have constant access to encrypted data as needed for patient care.
4. **Scalability**
    - The system should be scalable to handle a large volume of medical images, supporting the demands of healthcare facilities of varying sizes without degradation in performance.
5. **Usability**
    - The interface should be intuitive, enabling healthcare providers to encrypt and decrypt images with ease, minimizing the need for extensive training.
6. **Compliance**
    - The system must comply with healthcare data protection standards and regulations, such as HIPAA, GDPR, or relevant national privacy laws, to ensure lawful handling of sensitive patient information.
7. **Maintainability**
    - The system should be designed for easy maintenance and updates, allowing quick adaptations to emerging encryption technologies or changes in healthcare standards.
8. **Data Quality Preservation**
    - The encryption process should ensure that the diagnostic quality of medical images is preserved, avoiding any loss of clarity or detail essential for medical analysis.
9. **Interoperability**
    - The system should be compatible with existing healthcare infrastructure, such as PACS (Picture Archiving and Communication Systems) and EMRs (Electronic Medical Records), allowing seamless integration into healthcare workflows.
10. **Scalability and Flexibility**
    - The system should be adaptable to different healthcare settings, from small clinics to large hospitals, and able to handle varying workloads without affecting performance
    -

HARDWARE AND SOFTWARE REQUIREMENTS:

Hardware requirements

- System : intel i3/i5 2.4 GHz
- Hard Disk : 500 GB
- Ram : 4/8 GB

Software requirements

- Operating system : Windows XP/ Windows 7
- Software Tool : Open CV – Python
- Coding Language: Python. HTML, Css, Js

SOFTWARE IMPLEMENTATION

4.1 Python IDE

Python is an easy to learn, powerful programming language. It has efficient high-level data structures and a simple but effective approach to object-oriented programming. Python's elegant syntax and dynamic typing, together with its interpreted nature, make it an ideal language for scripting and rapid application development in many areas on most platforms. The Python interpreter is easily extended with new functions and data types implemented in C or C++ (or other languages callable from C). Python is also suitable as an extension language for customizable applications.

4.2 OpenCV

OpenCV is a library of programming functions mainly aimed at real-time computer vision. It has a modular structure, which means that the package includes several shared or static libraries. We are using image processing module that includes linear and non-linear image filtering, geometrical image transformations (resize, affine and perspective warping, and

generic table-based remapping), color space conversion, histograms, and so on. Our project includes libraries such as Viola-Jones or Haar classifier, LBPH (Lower Binary Pattern histogram) face recognizer, Histogram of oriented gradients (HOG).

4.3 OpenCV-Python

Python is a general purpose programming language started by **Guido van Rossum**, which became very popular in short time mainly because of its simplicity and code readability. It enables the programmer to express his ideas in fewer lines of code without reducing any readability.

Compared to other languages like C/C++, Python is slower. But another important feature of Python is that it can be easily extended with C/C++. This feature helps us to write computationally intensive codes in C/C++ and create a Python wrapper for it so that we can use these wrappers as Python modules. This gives us two advantages: first, our code is as fast as original C/C++ code (since it is the actual C++ code working in background) and second, it is very easy to code in Python. This is how OpenCV-Python works, it is a Python wrapper around original C++ implementation. And the support of Numpy makes the task more easier. Numpy is a highly optimized library for numerical operations. It gives a MATLAB-style syntax. All the OpenCV array structures are converted to-and-from Numpy arrays. So whatever operations you can do in Numpy, you can combine it with OpenCV, which increases number of weapons in your arsenal. Besides that, several other libraries like SciPy, Matplotlib which supports Numpy can be used with this. So OpenCV-Python is an appropriate tool for fast prototyping of computer vision problems.

OpenCV-Python working

OpenCV introduces a new set of tutorials which will guide you through various functions available in OpenCV-Python**.** This guide is mainly focused on OpenCV 3.x version (although most of the tutorials will work with OpenCV 2.x also). A prior knowledge on Python and Numpy is required before starting because they won't be covered in this guide. Especially, a good knowledge on Numpy is must to write optimized codes in OpenCV-Python.

This tutorial has been started by *Abid Rahman K.* as part of Google Summer of Code 2013 program, under the guidance of *Alexander Mordvintsev*.

OpenCV Needs us:

Since OpenCV is an open source initiative, all are welcome to make contributions to this library. And it is same for this tutorial also. So, if you find any mistake in this tutorial (whether it be a small spelling mistake or a big error in code or concepts, whatever), feel free to correct it. And that will be a good task for freshers who begin to contribute to open source projects. Just fork the OpenCV in github, make necessary corrections and send a pull request to OpenCV. OpenCV developers will check your pull request, give you important feedback and once it passes the approval of the reviewer, it will be merged to OpenCV. Then you become a open source contributor. Similar is the case with other tutorials, documentation etc.

As new modules are added to OpenCV-Python, this tutorial will have to be expanded. So those who knows about particular algorithm can write up a tutorial which includes a basic theory of the algorithm and a code showing basic usage of the algorithm and submit it to OpenCV.

1. Getting Started with Images

Goals

- Here, you will learn how to read an image, how to display it and how to save it back
- You will learn these functions : **cv2.imread()**, **cv2.imshow()** , **cv2.imwrite()**
- Optionally, you will learn how to display images with Matplotlib

Using OpenCV

Read an image

Use the function **cv2.imread()** to read an image. The image should be in the working directory or a full path of image should be given.

Second argument is a flag which specifies the way image should be read.

- cv2.IMREAD_COLOR : Loads a color image. Any transparency of image will be neglected. It is the default flag.
- cv2.IMREAD_GRAYSCALE : Loads image in grayscale mode
- cv2.IMREAD_UNCHANGED : Loads image as such including alpha channel

Display an image

Use the function **cv2.imshow()** to display an image in a window. The window automatically fits to the image size.

First argument is a window name which is a string. second argument is our image. You can create as many windows as you wish, but with different window n**cv2.waitKey()** is a keyboard binding function. Its argument is the time in milliseconds. The function waits for specified milliseconds for any keyboard event. If you press any key in that time, the program continues. If **0** is passed, it waits indefinitely for a key stroke. It can also be set to detect specific key strokes like, if key *a* is pressed etc which we will discuss below.

cv2.destroyAllWindows() simply destroys all the windows we created. If you want to destroy any specific window, use the function **cv2.destroyWindow()** where you pass the exact window name as the argument.

4.4 Image processing module

Purpose of Image processing

The purpose of image processing is divided into 5 groups. They are:

1. Visualization- Observe the objects that are not visible.
2. Image sharpening and restoration- To create a better image.
3. Image retrieval- Seek for the image of interest.
4. Measurement of pattern– Measures various objects in an image.
5. Image Recognition– Distinguish the objects in an image

Tools and Technologies:

1. **Flask**: A lightweight web framework for Python used to devel op the backend of the web application, handling HTTP requests, and routing.
2. **OpenCV**: A computer vision library used for processing medical images, including tasks like encryption and decryption while ensuring image quality.

3. **PyCryptodome**: A Python library for cryptography, specifically for encryption and decryption operations, providing tools like AES, DES, and other algorithms for securing the images.
4. **SQLite**: A lightweight, serverless database used for storing data such as medical images, encrypted images, and decryption keys in the local environment.
5. **HTML**: A markup language for structuring the front-end web pages, including the user interface for interacting with the system.
6. **CSS**: A style sheet language used to design the front-end of the web application, making it visually appealing.
7. **JavaScript (JS)**: A scripting language for adding interactivity and dynamic features to the web pages, enhancing the user experience.

Programming Languages:

1. **Python**: The primary programming language used for developing the backend logic of the encryption and decryption system, along with integrating other tools like Flask, OpenCV, and PyCryptodome.

Frameworks and Libraries:

- **Flask**: Web framework for Python.
- **OpenCV**: For image processing and manipulation.
- **PyCryptodome**: For cryptographic functions.
- **SQLite**: For database management in the local environment.